This is a massive, highly structured, and ambitious Google Apps Script framework. You've implemented several advanced patterns that are rarely seen in typical Apps Script projects, including in-memory lookups, property-based payload hashing, and built-in execution timing logs.

Here is a review of the script's efficacy, correctness, runtime speed, Google Workspace quota safety, and overall architecture.

I. Runtime Speed & Apps Script Quota Safety

Strengths:

Batch Operations: You correctly use getValues() and setValues() to interact with the sheet in bulk, which is the golden rule of Apps Script performance.

Intelligent Row Deletion: deleteRowNumberBatches_ is exceptionally well-written. By sorting the target rows descending and grouping contiguous row deletions into a single deleteRows(start, count) call, you save dozens (or hundreds) of expensive API calls.

Template Format Signatures: Using PropertiesService to store a hashed state of the dashboard's format (compactTemplateFormatSignature_) to skip redundant full-rebuilds is an excellent optimization. You also intelligently capped this at 8,500 characters, keeping it safely under Google's 9KB per-property limit.

Runtime Caching: Your use of global constant objects (ML_MONTHLY_SHEET_LOOKUP_CACHE_, ML_HEADER_CACHE_) safely limits redundant getSheetByName() and getRange().getValues() calls during a single script execution.

Risks & Recommendations:

Execution Time Limit (6 minutes): Rebuilding all templates (createOrRefreshAllReportTemplates) simultaneously applies heavy base formatting, conditional formats, and data validations. If the RFF_FORCE_FULL_TEMPLATE_REBUILD flag is toggled on, this single function could threaten the 6-minute execution limit.

Row Height / Column Width Loops: In functions like applyColumnWidthsInRuns_, you loop through columns to set widths. Even with run-grouping, interacting with the Sheet UI grid structure is significantly slower than data operations. Ensure you only resize columns/rows when absolutely necessary, relying on the RFF_FAST_TEMPLATE_REFRESH bypass whenever possible.

II. Efficacy & Correctness

Strengths:

Data Parsing/Coercion: coerceToValidDate_ handles serial spreadsheet dates, standard JS dates, string parsing, and edge cases beautifully.

Composite Key Hashing: buildSourceHashForRows_ successfully creates an MD5 digest of row data, which is an extremely reliable way to check for row mutations without doing a cell-by-cell loop.

Risks & Recommendations:

Call Stack Exceeded Risk: In appendMasterListChangeLog_, you calculate the maximum row length using Math.max.apply(null, rows.map(function(row) { return row.length; }));. In the V8 engine, passing arrays into .apply() risks a "Maximum call stack size exceeded" error if the array has more than ~65,000 elements. While your change logs may never reach this size in a single burst, it's safer to use a reduce loop:

JavaScript

```
const width = rows.reduce((max, row) => Math.max(max, row.length), 0);
```

Feature Stubs: shouldUseCorrectedIndividualTemplateBuild_, hideMasterListSourceColumns_, and showMasterListSourceColumns_ currently return false or 0 outright. Ensure these stubs are intentional and don't mask intended functionality that hasn't been fully wired up.

III. Architecture Compliance

Strengths:

Decoupled Configuration: Pulling the template layout, column behaviors, and formatting rules from a "Format Dashboard" sheet rather than hardcoding them inside the Javascript logic makes the framework incredibly resilient and accessible to non-technical maintainers.

Defensive Design: You make excellent use of fallback parameters (e.g., Number(value || 25) and robust falsy checking). Functions like ensureOutputSheetHasFormattedRows_ ensure structure exists before data is dumped.

Health and Audit Logging: The inclusion of runFrameworkHealthCheck and writeCombinedFrameworkTimingReport_ elevates this from a simple script to enterprise-grade automation. It self-diagnoses its own API latencies and missing function dependencies.

Risks & Recommendations:

Memory Footprint: In splitRawDataRowsIntoActiveAndDisenrolled_ and large mapping scripts, you are duplicating large 2D arrays via .slice() and mapping them into new arrays. The Apps Script V8 engine has roughly ~50MB of usable memory. If Demo P or Raw Data grows to 20,000+ rows with 40+ columns, keeping multiple copies of these arrays in memory (previous demo data, current demo data, mapping layers) might hit memory ceilings. Where possible, map array values in place or rely heavily on indexing/pointers.

Overall, this is a highly sophisticated, production-ready framework. The caching mechanisms, batching strategies, and signature validations show a deep understanding of Google Apps Script constraints.

Are there specific sheets or workflows within this framework that are currently experiencing timeouts or performance bottlenecks?